

MessageFormat 2 — Implementation Details

- Author: Alexey Lyakhov

Companion to `messageformat-v2-research.md` (background) and `messageformat-v2-architecture.md` (crate/type layout).

This document is the **execution plan**: a sequence of red/green/refactor commit phases, the testing strategy, the tooling/CI wiring, and the quality-gate contract that every phase must pass before merge.

1. Testing strategy

Test tiers

1. **Unit tests** — inline `#[test]` under each module for single-responsibility helpers (lexer, grammar, validator rules, selector comparator). Small, fast, no data provider.
2. **Integration tests** — `components/experimental/tests/messageformat/*.rs` entries wired through `[[test]]` in `Cargo.toml`, with real data providers enabled by `compiled_data`. Each scenario is a small piece of API that can be used by downstream code.
3. **Conformance tests** — a data-driven runner over the vendored WG fixture suite (`tests/messageformat/fixtures/tests/**/*.json`). One Rust test function per fixture file, with one panic per failing sub-case so failures are granular in CI output.
4. **Doctests** — every public type/method has a compilable example in its rustdoc; they run as part of `cargo make ci-job-doc`.
5. **Negative tests** — syntax-errors and data-model-errors fixtures assert that `Message::parse / validate` return the right error tag.
6. **Benchmarks** (deferred to phase 11) — criterion benches for parse, build, and format throughput. Registered in `Cargo.toml` `[[bench]]`, gated by `cfg(not(target_arch = "wasm32"))`.

TDD loop

Every feature lands as a triplet of commits unless trivially small:

- **red**: add the failing test (or `#[ignore]` the conformance line). Commit compiles; test fails/ignored.
- **green**: smallest implementation that makes the test pass. Commit compiles, all targeted tests pass, lints pass.

- **refactor**: clean up — rename, deduplicate, tighten types, extract helpers. No behavior change; tests still pass.

Each commit is its own atomic PR-ready unit: lints, fmt, and cargo make quick must pass. Commit messages follow the existing convention seen in `git log` (imperative, area-prefixed):

```
"messageformat: parse simple-message (red)" .
```

Fixture vendoring

- One-shot sync from the working-group repo at a pinned commit SHA.
- Vendor path:
`components/experimental/tests/messageformat/fixtures/` .
- The SHA is recorded in `fixtures/README.md`; a `cargo make sync-mf2-tests` target in `tools/make/tests.toml` runs a `rsync` from `/Users/alexeylyakhov/projects/message-format-wg/test/` (filtered to `tests/`, `schemas/`, `README.md`, `LICENSE`), then updates the SHA file from `git -C message-format-wg rev-parse HEAD` .
- Fixtures are checked into the repo, so contributors and CI need no network access. Updates are deliberate, reviewed commits.

Conformance runner shape

```
// tests/messageformat/conformance.rs
#[derive(serde::Deserialize)]
struct Suite { tests: Vec<Case>, defaultTestProperties: Option<Defaults> }

#[derive(serde::Deserialize)]
struct Case { src: String, params: Option<Vec<Param>>, exp: Option<String>,
              expErrors: Option<Vec<ErrTag>>, expParts: Option<Vec<PartTag>>,
              locale: Option<String>, only: Option<bool>, /* ... */ }

#[test]
fn syntax_conformance() { run("fixtures/tests/syntax.json"); }
#[test]
fn syntax_errors_conformance() { run("fixtures/tests/syntax-errors.json"); }
#[test]
fn data_model_errors_conformance() { run("fixtures/tests/data-model-errors.json"); }
#[test]
fn pattern_selection_conformance() { run("fixtures/tests/pattern-selection.json"); }
#[test]
fn fallback_conformance() { run("fixtures/tests/fallback.json"); }
#[test]
fn u_options_conformance() { run("fixtures/tests/u-options.json"); }
#[test]
fn bidi_conformance() { run("fixtures/tests/bidi.json"); }
#[test]
fn functions_string_conformance() { run("fixtures/tests/functions/string.json"); }
#[test]
fn functions_number_conformance() { run("fixtures/tests/functions/number.json"); }
#[test]
fn functions_integer_conformance() { run("fixtures/tests/functions/integer.json"); }
#[cfg(feature = "unstable")]
#[test]
fn functions_datetime_conformance() { run("fixtures/tests/functions/datetime.json"); }
// ...
```

`run()` builds a `MessageFormatter` per case, calls `format_to_string` and `format_to_parts`, compares against `exp` / `expParts`, and checks that the error tags match `expErrors`. Failed cases are collected and reported via a single assertion after the loop, so one run shows every regression.

Each test starts out as a `red` step where the called fixture is empty (or the runner recognizes `only = true` on a single passing case). Cases are enabled in batches as phases land.

2. Quality gates every commit must pass

Gate	Command	Why
Format	<code>cargo fmt --check</code>	Standard.
Clippy	<code>cargo clippy --workspace --all-features --all-targets -- -D warnings</code>	Matches <code>ci-job-clippy</code> .
Build (no features)	<code>cargo check -p icu_experimental --no-default-features</code>	Verifies <code>no_std</code> compatibility.
Build (all features)	<code>cargo check -p icu_experimental --all-features</code>	Catches conflicting feature-gates.
Unit+integration tests	<code>cargo test -p icu_experimental --all-features</code>	Every test passes.
Conformance	<code>cargo test -p icu_experimental --test messageformat_test</code>	Subset, pinned.
Doc build	<code>cargo doc -p icu_experimental --all-features --no-deps</code>	Matches <code>ci-job-doc</code> .
Workspace quick	<code>cargo make quick</code>	Aggregate: <code>fmt</code> + <code>clippy</code> + <code>check</code> + <code>depcheck</code> + <code>doc</code> .

A commit landing as "green" MUST have all of the above passing. Only "red" commits are allowed to have failing tests, and only the tests introduced in that commit.

All `pub` types carry `Debug` (workspace lint enforces this). All `pub enum`s are `#[non_exhaustive]` (workspace lint enforces this). `lib.rs`-style annotations at `components/experimental/src/messageformat/mod.rs` only `#![allow(...)]` what the other experimental submodules allow, nothing more (no bespoke relaxation).

3. Phase-by-phase commit plan

Each phase is a small, reviewable slice. Phases are ordered so every commit compiles and tests pass at the "green" step. Phases 1-8 cover the v1 stable surface (`:string` + `:number` + `:integer`). Phases 9-11 cover draft functions, bidi polish, and performance.

Phase 0 — scaffolding

Goal: the module exists, is reachable from `icu_experimental`, has the canonical boilerplate and a placeholder public API.

Commits:

1. `messageformat`: scaffold submodule (green)
 - Create `components/experimental/src/messageformat/mod.rs` with the license header, sub-module relaxation allows, and the single-line crate doc.
 - Add `pub mod messageformat;` to `components/experimental/src/lib.rs` alphabetically.
 - Create empty stubs: `ast.rs`, `parser.rs`, `formatter.rs`, `error.rs` — each with license header and `//! TODO` doc.
 - Create `tests/messageformat/tests.rs` entry stub + `[[test]]` registration in `components/experimental/Cargo.toml`.
 - Create `tests/messageformat/fixtures/README.md` with the pinned upstream SHA and the sync command.
 - Gate tests: `required-features = []` initially.

No red step for pure scaffolding; it's infrastructure. Verified by `cargo make quick` passing.

Phase 1 — AST types + serde round-trip

Goal: types from `architecture.md` §4 are defined, derivable, and round-trip through `serde_json`.

1. `messageformat`: add AST types (red)
 - `tests/messageformat/ast_roundtrip.rs` with an `#[ignore]` -free test that parses the spec JSON Schema sample into the AST and serializes it back, asserting semantic equivalence. The test does not compile yet (types missing).
2. `messageformat`: add AST types (green)
 - `ast.rs` populated with every type from `architecture` §4.
 - `serde` derives behind `#[cfg(feature = "serde")]`.
 - `datagen` derives behind `#[cfg(feature = "datagen")]`.
 - Test passes with `--features serde`.
3. `messageformat`: AST refactor (refactor)
 - Split `ast.rs` into `ast/mod.rs` + `ast/expression.rs` + `ast/markup.rs` if the file exceeds ~400 lines.
 - Extract shared `Identifier::new_checked` helper.
 - No behavior change.

Phase 2 — lexer

Goal: character-level tokenizer covering whitespace, bidi controls, keywords (`.input` / `.local` / `.match`), punctuation (`{ }` `{ }` `|` `\\`), literals (unquoted, quoted), names. Zero allocation for fixed tokens.

1. messageformat: lex simple tokens (red)
 - `parser/lexer.rs` test table: `("hello", [Text("hello")])`, `("${x}", [OpenBrace, Dollar, Name("x"), CloseBrace])`, etc. Compiles but fails (lexer not implemented).
2. messageformat: lex simple tokens (green)
 - Lexer passes plain text, variables, braces, pipes, backslash escape, keyword detection.
3. messageformat: lex names per UAX#31 profile (red + green)
 - Add tests for Unicode names (Greek, CJK, combining marks).
 - Implement via `icu_properties` `Id_Start` / `Id_Continue` where feasible, plus explicit bidi-skip rules from ABNF.
4. messageformat: lexer refactor
 - Extract peek/bump helpers; document invariants.

Phase 3 — parser

Goal: parse simple-message and complex-message into AST. Produces `ParseError` with byte offsets for malformed input.

1. messageformat: parse simple pattern (red)
 - `tests/messageformat/parser.rs`: `"Hello, ${u}!"` parses to `PatternMessage` with three elements.
2. messageformat: parse simple pattern (green)
 - Hand-written recursive descent. No declarations yet. Panics on unknown tokens — flagged for later.
3. messageformat: parse declarations (red + green)
 - `.input ${x :fn o=v}` and `.local $y = ${x :fn}` produce the expected AST.
4. messageformat: parse matchers (red + green)
 - `.match` + variants, including `*`.
5. messageformat: parse markup (red + green)
 - `{#tag}`, `{#tag /}`, `{/tag}`, nested options/attributes.
6. messageformat: parse attributes and function options (red + green)
 - Full coverage of attribute/option parsing, including variable references as option values.
7. messageformat: parser error recovery (red + green)
 - `ParseError::Syntax { offset, message }` at precise byte positions; hook into the syntax-errors conformance fixture.
 - Enable `syntax_conformance` and `syntax_errors_conformance` fixtures.
8. messageformat: parser refactor

- Pull the peek/bump/expect boilerplate into helpers; document the error-offset invariant; tighten types (`Cow<'src, str>` for text runs).

Phase 4 — validator

Goal: post-parse data-model checks; emit `ValidationError` enums matching the spec's error taxonomy.

1. messageformat: validate missing fallback variant (red + green)
 - Conformance slice: `data-model-errors.json` cases tagged `missing-fallback-variant` .
2. messageformat: validate duplicate declaration (red + green)
3. messageformat: validate duplicate variant (red + green)
4. messageformat: validate duplicate option name (red + green)
5. messageformat: validate variant key mismatch (red + green)
6. messageformat: validate missing selector annotation (red + green)
 - Walk declarations to check that every selector is transitively function-annotated.
 - At this point `data_model_errors_conformance` is fully enabled.
7. messageformat: validator refactor
 - Consolidate visitor; produce a typed `ValidatedMessage` wrapper so downstream code cannot consume an unvalidated `Message` .

Phase 5 — formatter skeleton + `:string`

Goal: end-to-end: parse → validate → format a pattern with literal text and a `{ $var }` placeholder, using the default `:string` function.

1. messageformat: `ResolvedValue` + `Writeable` scaffolding (red + green)
2. messageformat: `InputValues` trait + `BTreeMap` impl (red + green)
3. messageformat: resolver for input variables (red + green)
 - Hello, `{ $user }!` with `user = "Ada"` → Hello, Ada! .
4. messageformat: `:string` function handler (red + green)
 - Register the built-in `:string` handler.
 - Simple format path, no selection yet.
 - Unresolved-variable path emits fallback value `{ $name }` and records a `FormatError::UnresolvedVariable` without panicking.
5. messageformat: fallback substitution rules (red + green)
 - Enable `fallback_conformance` and the subset of `functions/string.json` covering format (not select).
6. messageformat: formatter refactor
 - Encapsulate `FormatCtx` (locale, direction, registry, error sink).

Phase 6 — pattern selection + `:string` selector

Goal: `.match` works end-to-end for a string selector.

1. `messageformat`: Selector trait + Matcher algorithm (red)
 - Unit tests for the selection algorithm with a hand-rolled `:test:select` that reproduces the spec's table.
2. `messageformat`: Selector trait + Matcher algorithm (green)
 - Implementation per architecture §6. All-catchall fallback works.
3. `messageformat`: `:string` selection with NFC compare (red + green)
 - Uses `icu_normalizer::ComposingNormalizer` to normalize both selector value and variant key before equality.
4. `messageformat`: enable `pattern_selection_conformance` (green)
 - Wire the `:test:select` harness function.
 - Turn on `pattern_selection_conformance` (any remaining failures become ignored individual sub-cases; add a comment linking to a tracking issue).
5. `messageformat`: selector refactor
 - Extract `VariantPicker` struct with unit tests for tie-breaking.

Phase 7 — `:number` and `:integer`

Goal: pluralization and numeric formatting, the core MF2 value-add.

1. `messageformat`: number operand coercion (red + green)
 - Accept `InputValue::Number(Decimal)`, integers, stringified numerics; reject others with `FunctionError::BadOperand`.
2. `messageformat`: `:number` formatter via `icu_decimal` (red + green)
3. `messageformat`: `:number` option parsing (red + green)
 - `signDisplay`, `useGrouping`, `digit sizes`, rounding. Invalid values → `FunctionError::BadOption`.
4. `messageformat`: `:number` plural selector via `icu_plurals` (red + green)
 - Both `select=plural` and `select=ordinal`; `select=exact` uses string equality on the formatted-integer form.
5. `messageformat`: `:integer` (red + green)
 - Trivial delegation to `:number` with forced integer digit options.
6. `messageformat`: enable `functions_number_conformance` / `functions_integer_conformance`
 - All cases in the stable subset enabled.
7. `messageformat`: number refactor
 - Dedupe option parsing between `:number` and `:integer`.

Phase 8 — `u:` namespace + bidi basics

Goal: `u:id`, `u:dir`, bidi isolation in concatenated output.

1. `messageformat`: `u:id` propagation (red + green)
 - `u:id` is stripped from options before handler call, preserved in the resolved part.
2. `messageformat`: `u:dir` override (red + green)
 - Valid values; emits `FunctionError::BadOption` otherwise.
3. `messageformat`: bidi isolation (red + green)

- Default on; wraps placeholders with direction mismatch in LRI/RLI/FSI + PDI.
 - `bidi_isolation(false)` disables.
 - Enables `u_options_conformance` and `bidi_conformance`.
4. `messageformat`: `bidi` refactor
- Factor direction detection into `bidi::resolve_direction`.

Phase 9 — format-to-parts

Goal: structured output API mirroring `messageformat.formatToParts`.

1. `messageformat`: `FormattedPart` type + parts collector (red + green)
2. `messageformat`: `format_to_parts` for string/number (red + green)
 - Enable the `expParts` assertions in existing conformance fixtures.
3. `messageformat`: `format_to_parts` refactor

Phase 10 — draft datetime functions

Gated behind `feature = "unstable"`.

1. `messageformat`: `:date` via `icu_datetime` (red + green, unstable)
2. `messageformat`: `:time` via `icu_datetime` (red + green, unstable)
3. `messageformat`: `:datetime` via `icu_datetime` (red + green, unstable)
4. `messageformat`: enable `functions_datetime_conformance` (unstable)

Phase 11 — performance + docs

1. `messageformat`: criterion benches for `parse+format`
 - `components/experimental/benches/messageformat/*.rs`.
 - Tracks: tiny simple message, 1KB pattern with 10 placeholders, medium matcher with 20 variants.
2. `messageformat`: parser zero-copy refinement
 - Replace `String` with `Cow<'src, str>` where profiling shows it saves allocation without regressions.
3. `messageformat`: crate-level docs + tutorial
 - Flesh out the module-level doc with runnable examples (doctests).
 - Add `tutorials/messageformat_v2.md` under `/tutorials/`.
 - Update `CHANGELOG.md` for `icu_experimental 0.6.0`.

Phase 12 — graduation (OUT OF SCOPE OF V1, DOCUMENTED)

When the surface is proven by external users:

- Move `components/experimental/src/messageformat/**` → `components/messageformat/**`.
- Create `provider/data/messageformat/` baked-data crate if custom markers exist at that point.
- Add FFI layer in `ffi/capi/src/messageformat.rs` via `Diplomat`.
- Add re-export in `components/icu/src/lib.rs`.
- Register in workspace `Cargo.toml` members and `workspace.dependencies`.

- Bump to `icu_messageformat = 0.1.0` .

4. Lint & style contract (must match every commit)

- Every source file starts with the ICU4X license header — copy verbatim from `components/experimental/src/lib.rs` .
- Module root has the existing experimental relaxation block:


```
#![allow(clippy::panic, clippy::indexing_slicing, clippy::unwrap_used, missing_docs)]
```

 (keep the allows scoped to the submodule, don't add new ones).
- Every public enum: `#[non_exhaustive] + Debug` .
- Every public struct: `Debug` , `#[non_exhaustive]` unless the fields are truly stable.
- Errors: `displaydoc::Display` (no `thiserror` dep).
- No `std` -only types in public API. Use `alloc::*` and `core::*` everywhere.
- No `panic!` , `unwrap` , `expect` , or `[i]` slicing outside of tests.
The submodule `allow` is a grandfather clause for experimental work — new code SHOULD still avoid these.
- No direct `println!` / `eprintln!` / `dbg!` . Use the `log` feature if needed (already available in `icu_experimental`).
- Clippy pedantic? No — match the workspace lint table exactly.
Specifically respected: `doc_markdown` , `missing_fields_in_debug` , `todo` , `or_fun_call` , `trivially_copy_pass_by_ref` = `deny` , `exhaustive_enums` = `deny` .

5. CI integration

- No change to `.github/workflows` is required for the experimental submodule: tests run under `cargo test -p icu_experimental --all-features` , which is already part of `ci-job-test` .
- Add entries in `components/experimental/Cargo.toml` :

```
[[test]]
name = "messageformat_test"
path = "tests/messageformat/tests.rs"
required-features = ["serde"]      # serde_json for fixture parsing
```

- Add a makefile subtask in `tools/make/tests.toml` :

```
[tasks.sync-mf2-tests]
description = "Re-vendor the MessageFormat 2 WG conformance fixtures."
command = "bash"
args = ["tools/scripts/sync-mf2-tests.sh"]
```

- The bench target (phase 11) registers under:

```
[[bench]]
name = "messageformat"
path = "benches/messageformat/bench.rs"
harness = false
```

6. Documentation deliverables

Per phase, per file:

File	Purpose
components/experimental/src/messageformat/mod.rs	Module doc: scope, quickstart, links to spec.
Every pub item	Rustdoc with at least one runnable example (doctest).
tutorials/messageformat_v2.md	End-to-end walk-through for users migrating from MF1. Added in phase 11.
CHANGELOG.md	Entry under <code>icu_experimental</code> per minor version.
components/experimental/README.md	Auto-generated from <code>lib.rs</code> via <code>cargo-rdme</code> ; regenerated in CI.

`documents/design/messageformat_v2.md` — a short design record
referencing the three `messageformat-v2-*.md` files. Added in phase 0
alongside the scaffolding commit so reviewers have context from the
start.

7. Risks and mitigations

Risk	Mitigation
Spec clarifications post-LDML-46.1	Pin the WG fixture SHA; surface discrepancies as <code>#[ignore]</code> d conformance cases linked to spec issues. Re-sync fixtures on a cadence.
<code>:datetime</code> options churn (Draft status)	Keep the draft functions under <code>feature = "unstable"</code> ; release notes warn consumers.
Custom functions escape hatch becomes over-permissive	Require <code>Send + Sync</code> on <code>FunctionHandler</code> from day 1; limit trait surface; revisit after real users.
Parse-time locale-dependent rules (NFC, bidi) spreading into the parser	Keep the parser locale-free; all locale-sensitive work lives in <code>formatter/resolver</code> . Enforced by not passing <code>LocalePreferences</code> into <code>parser::parse</code> .

Risk	Mitigation
Performance regressions on large matchers	Add benchmarks in phase 11 before merging more features; criterion output tracked in CI via <code>ci-job-test</code> .

8. Definition of "v1 done"

All of the following pass, as a single green state, on `main` merge of the final phase-8 PR:

- ☐ `cargo make quick` green.
- ☐ `cargo make ci-job-test` green including all enabled MF2 conformance fixtures.
- ☐ 100% of `syntax.json` , `syntax-errors.json` , `data-model-errors.json` , `fallback.json` , `pattern-selection.json` , `u-options.json` , `bidirectional.json` , `functions/string.json` , `functions/number.json` , `functions/integer.json` fixtures pass.
- ☐ Draft `:date / :time / :datetime` pass behind `--features unstable` or are explicitly `#[ignore]` d with tracking issues.
- ☐ `cargo doc --all-features --no-deps` builds cleanly for `icu_experimental` .
- ☐ CHANGELOG entry, module-level doc with runnable doctests, and a tutorial under `/tutorials/` .
- ☐ No new workspace lint violations.